

LLD – Saved Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Saved (Bookmark) module of OpenScholar. The module allows authenticated users to save (bookmark) papers for later access and manage their saved collection efficiently.

The design ensures idempotency, fast lookup, and scalability.

2. Scope

The Saved module is responsible for:

- Save (bookmark) a paper
- Remove a saved paper
- Retrieve list of saved papers
- Prevent duplicate saves

Out of Scope:

- Recommendation logic
 - Folder/collection grouping (future enhancement)
-

3. Data Model

saved_papers

- user_id (UUID)
- paper_id (UUID)
- created_at (TIMESTAMP)

Constraints:

- PRIMARY KEY (user_id, paper_id)
- FOREIGN KEY user_id → users.id
- FOREIGN KEY paper_id → papers.id

Indexes:

- INDEX on user_id
- INDEX on paper_id

4. API Contracts

4.1 Save Paper

Endpoint: POST /api/saved

Headers: Authorization: Bearer <jwt>

Request Body: { "paperId": "uuid" }

Response: { "message": "Paper saved successfully" }

Behavior:

- If already saved → return success (idempotent)

4.2 Remove Saved Paper

Endpoint: DELETE /api/saved

Headers: Authorization: Bearer <jwt>

Request Body: { "paperId": "uuid" }

Response: { "message": "Paper removed from saved" }

4.3 Get Saved Papers

Endpoint: GET /api/saved

Headers: Authorization: Bearer <jwt>

Query Parameters:

- page (default: 1)
- limit (default: 10)

Response: { "total": 25, "results": [{ "paperId": "uuid", "title": "Paper Title", "abstract": "Short abstract", "savedAt": "timestamp" }] }

5. Service Logic

5.1 savePaper(userId, paperId)

Steps: 1. Validate paper exists and is approved 2. Insert into saved_papers

- If conflict → ignore (ON CONFLICT DO NOTHING)
 - Return success
-

5.2 removeSavedPaper(userId, paperId)

Steps: 1. Delete from saved_papers where user_id and paper_id match 2. Return success

5.3 getSavedPapers(userId, page, limit)

Steps: 1. Query saved_papers by user_id 2. Join papers and latest paper_versions 3. Apply pagination 4. Return results

6. Validation Rules

- user must be authenticated
 - paper_id must exist
 - only approved papers can be saved
-

7. Security Considerations

- Users can only access their own saved papers
 - Validate JWT on all endpoints
-

8. Performance Considerations

- Use index on user_id for fast retrieval
 - Use composite PK to prevent duplicates
 - Join only required fields from papers and paper_versions
-

9. Edge Cases

- Saving already saved paper

- Removing non-existent saved record
 - Large number of saved papers (pagination required)
-

10. Prisma Model Mapping

```
model SavedPaper { userId String paperId String createdAt DateTime @default(now())
```

```
@@id([userId, paperId]) }
```

11. Folder Structure

```
src/ app/api/saved/route.ts
```

```
modules/saved/service.ts modules/saved/repository.ts
```

12. Implementation Notes

- Use Prisma upsert or create with skip duplicates
 - Avoid unnecessary joins for performance
 - Ensure API idempotency
-

13. Acceptance Criteria

- User can save a paper
 - User cannot duplicate save
 - User can remove saved paper
 - User can view saved papers with pagination
-

14. Future Enhancements

- Save folders/collections
 - Tagging saved papers
 - Sync across devices
-

End of Document